# Advanced Programming for Scientists

Lecture 5-1

**Versioning II: Collaboration and CI/CD**

**YAML: A Configuration Language**

v1.0

Prof. Mateusz Bawaj

A.D. 1308 unipg
UNIVERSITÀ DEGLI STUDI DI PERUGIA

# YAML = “YAML Ain’t Markup Language”

Purpose:

- Human-readable configuration files
- Widely used in scientific software infrastructure
- Common in CI/CD pipelines, container orchestration and experiment workflows

Typical uses in research computing:

- CI pipelines (GitHub Actions, GitLab CI)
- Docker Compose configurations
- Workflow engines
- Experiment configuration files

Key advantage: Configuration separated from code → easier reproducibility.

# Basic YAML Structure

YAML is based on key–value pairs and indentation.

Example:

```yaml
simulation:
  model: fabry_perot
  finesse: 450
  round_trip_time: 2e-6
```

Interpretation:

- **`simulation`** → section
- **`model`**, **`finesse`**, **`round_trip_time`** → parameters

Important rule:

**Indentation defines structure** (no {} like JSON)

For indentation **use spaces, not tabs**.

# Lists in YAML

Lists are created using "–".

Example:

```
dependencies:
  - numpy
  - scipy
  - matplotlib
```

This corresponds to: `dependencies = [numpy, scipy, matplotlib]`

# Nested Structures

YAML allows hierarchical configuration.

Example:

```yaml
experiment:
  cavity:
    length: 3.0
    finesse: 450
  laser:
    wavelength: 1064e-9
    power: 1.0
```

Interpretation:

1) **experiment**
   1) **cavity parameters**
      1) **length**
      2) **finesse**
   2) **laser parameters**
      1) **wavelength**
      2) **power**

This is useful for expressing complex structures.

# YAML vs JSON

Example JSON and equivalent YAML:

```
{
 "laser": {
   "wavelength": 1064e-9,
   "power": 1.0
 }
}
```

```
laser:
  wavelength: 1064e-9
  power: 1.0
```

Advantages of YAML:

- easier to read
- less syntax clutter
- supports comments

# Comments in YAML

Comments start with "#".

```yaml
simulation:
  model: fabry_perot    # cavity model
  steps: 100000         # simulation duration
```

Block comments can be obtained by prefixing each line with # individually.

# YAML in Continuous Integration

Example fragment of a GitHub Actions pipeline:

```yaml
name: CI

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
```

Meaning:

- pipeline name → `CI`
- triggered when → `push` or `pull_request`
- job executed on → `ubuntu-latest`

The YAML file **describes workflow execution**.

# Best Practices for YAML

Good practices:

- keep files small and modular
- document parameters with comments
- avoid deep nesting
- use consistent indentation (2 spaces)

Scientific recommendation:

Store **experiment configuration in YAML** rather than hard-coding parameters.

Example file: `simulation.yaml` instead of editing the Python script.

# Loading Parameters from a YAML File in Python

Simulation parameters are stored in a YAML file and loaded at runtime.

Example configuration file **`simulation.yaml`**:

```yaml
cavity:
  length: 3.0
  finesse: 450
laser:
  wavelength: 1064e-9
  power: 1.0
```

Library required:

```
$ pip install pyyaml
```

```python
import yaml

with open("simulation.yaml", "r") as f:
    config = yaml.safe_load(f)

length = config["cavity"]["length"]
finesse = config["cavity"]["finesse"]
power = config["laser"]["power"]

print("Cavity length:", length)
```

# Loading Parameters from a YAML File in C++

In C++ scientific software, configuration files can also be used to separate experiment parameters from compiled code.

```cpp
#include <iostream>
#include <yaml-cpp/yaml.h>

int main() {
    YAML::Node config = YAML::LoadFile("simulation.yaml");

    double length = config["cavity"]["length"].as<double>();
    int finesse = config["cavity"]["finesse"].as<int>();
    double power = config["laser"]["power"].as<double>();

    std::cout << "Cavity length: " << length << std::endl;
    std::cout << "Laser power: " << power << std::endl;

    return 0;
}
```

# Loading Parameters from a YAML File in C++

Reproducibility. Compilation example:

```
g++ simulation.cpp -lyaml-cpp -o simulation
```

Remember about the dependencies:

```
$ sudo apt install libyaml-cpp-dev
```

# Loading Parameters from a YAML File in C++

**Makefile**

```
# Compiler
CXX = g++
CXXFLAGS = -O2 -Wall
# Target
TARGET = simulation
# Source
SRC = simulation.cpp
# Libraries
LIBS = -lyaml-cpp
# Default rule
all: $(TARGET)
$(TARGET): $(SRC)
    $(CXX) $(CXXFLAGS) $(SRC) -o $
(TARGET) $(LIBS)
# Run example
run: $(TARGET)
    ./$(TARGET)
# Clean build artifacts
clean:
    rm -f $(TARGET)
```

Instead of providing compilation command, Makefile demonstrates a simple reproducible workflow for C/C++.

Build with:

**$ make**

Run with:

**$ make run**

There is no leading package manager for C libraries.

Read more here.

# Loading Parameters from a YAML File in C++

**Makefile**

```
# Compiler
CXX = g++
CXXFLAGS = -O2 -Wall
# Target
TARGET = simulation
# Source
SRC = simulation.cpp
# Libraries
LIBS = -lyaml-cpp
# Default rule
all: $(TARGET)
$(TARGET): $(SRC)
    $(CXX) $(CXXFLAGS) $(SRC) -o $
(TARGET) $(LIBS)
# Run example
run: $(TARGET)
    ./$(TARGET)
# Clean build artifacts
clean:
    rm -f $(TARGET)
```

Can we improve the reproducility of this result?

# Key Takeaways: YAML in Scientific Software

What YAML is

- YAML is a human-readable configuration format
- Widely used in modern software infrastructure
- Standard in CI/CD pipelines, workflow managers and containers

Core principles

- Structure defined by indentation
- Uses key–value pairs
- Supports lists and hierarchical structures
- Allows comments for documentation